AGENTIC AI · AWS COST OPTIMIZATION

The Chat-FinOps Agent

Watch the AWS bill, explain what changed, and surface waste with dollar-value savings — then act only after a business owner approves. All read-only by default.

gpt-4o	boto3	Streamlit
+ LANGCHAIN	AWS APIS	PORTAL + CHAT

Prepared by Aravind Baranitharan · AI Agents for Cloud & DevOps

Cloud spend leaks — quietly

Why the bill creeps up

- Idle and oversized resources keep billing after the project ends.
- Unattached EBS, unassociated IPs, old snapshots — invisible until the invoice.
- No single owner watches cost across accounts.

Why cleanup stalls

- Finding waste is manual, tedious, and easy to postpone.
- Deleting the wrong thing is risky — so nothing gets deleted.
- There's no safe, auditable path from "found it" to "fixed it".

> **The gap:** teams need continuous cost insight **and** a governed way to act on it — not another dashboard nobody reads.

See it, explain it, save it — safely

See
spend, trend, forecast

→

Explain
month-over-month, spikes

→

Save
waste + $/mo

→

Act
approved & guarded

Visibility

Forecast vs budget, trend, spend per account.

Analytics

Service breakdown, MoM movers, spike root-cause.

Optimization

Idle/unused resources with dollar savings.

Governance

Approve → guarded execute → audit.

Real waste, found live

$269/yr

Surfaced from a real AWS account in one read-only scan — no agents installed, nothing changed:

- **Idle EC2** — 0.5% CPU over 14 days → $15/mo
- **2 unassociated Elastic IPs** → $7.20/mo
- **Stale S3 bucket** (7.9 GB, 33d) → lifecycle candidate

chat-finops — savings scan

```
scan ▸ optimization_report()

[Idle EC2]      i-03b… (BotxStudio)
                avg CPU 0.5%/14d  → $15.00/mo
[Elastic IP]    98.91.66.209  not assoc → $3.60/mo
[Elastic IP]    3.7.109.82    not assoc → $3.60/mo
[Stale S3]      sessiontranslations 33d  → lifecycle

total ▸ $22.38/mo (~$268.56/yr)
```

Read-only. Same code runs on mock data for safe demos.

LangChain tools over your account

A LangChain (gpt-4o) agent chooses from a small set of read-only tools. Cost tools read `Cost Explorer`; detectors read CloudWatch metrics and resource describes. The model reasons — **your code** runs the AWS calls.

- Cost: spend, trend, forecast, month-over-month.
- Optimization: idle EC2, unused EBS/EIP, snapshots, stale S3.
- Each finding carries an estimated **$/mo saving**.

finops_agent.py

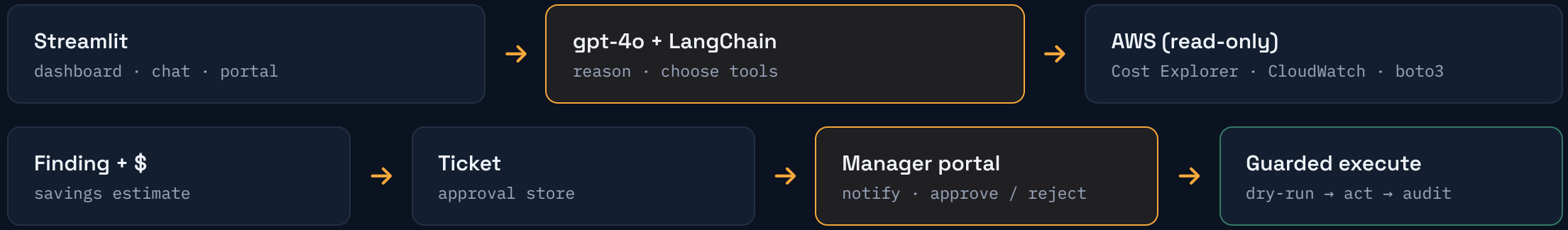
```python
from langchain.agents import create_agent

agent = create_agent(
    "openai:gpt-4o",
    [get_cost_summary,
     get_cost_changes,
     get_savings_opportunities])  # @tools

agent.invoke({"messages": [user_q]})
```

Read-only tools; the model never touches AWS directly.

Insight → approval → action



No email needed: requests become tickets in the portal; the approver gets an in-app notification, decides, and every step is recorded.

The waste it hunts

DETECTOR	SIGNAL	SAVING BASIS
Idle / low-use EC2	avg CPU below threshold (CloudWatch)	on-demand $/mo
Unattached EBS	volume state = available	$/GB-month
Unassociated Elastic IPs	no association	~$3.60/mo each
Old EBS snapshots	older than retention window	$/GB-month
Stale S3 buckets	no object updated in N days	Glacier/lifecycle

Extensible: Compute Optimizer rightsizing, unused NAT/ELB, Lambda & RDS tuning plug in as more detectors — same finding → savings → approval shape.

Know the bill before it lands

Forecast vs budget

Month-end projection from run-rate, with a breach alert.

Trend

6-month spend trajectory per account.

Spike root-cause

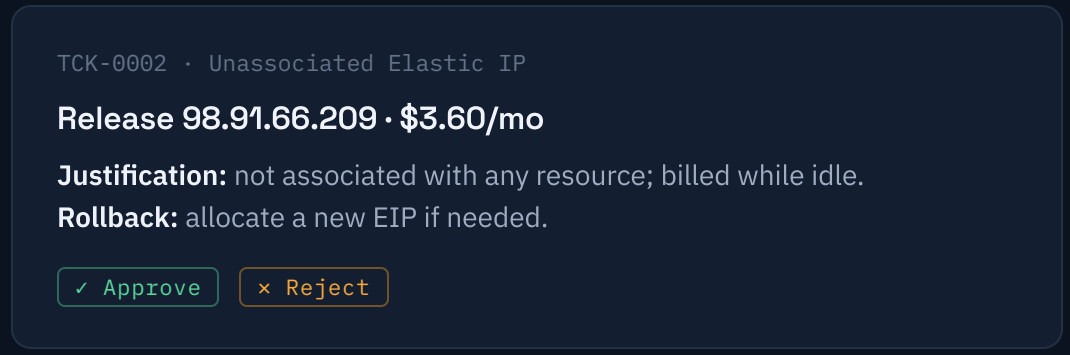
"RDS +38% — new read replica" — the driver, named.

Hybrid data: optimization is live today; the cost dashboard flips to real `Cost Explorer` once it's enabled (~24h), with clearly-labeled sample data until then.

The approval portal

An engineer raises a finding; it becomes a **ticket**. The business owner sees a notification, opens the portal, and reviews impact, justification and a rollback plan before deciding. No mail — the hierarchy approves in the portal.

- `notify` pending count = the manager's alert
- `approve / reject` role-gated decision
- **Audit** — who approved what, when, and the outcome



Requested by engineer · awaiting manager

Act only when approved

Approved
by the owner

→

Dry-run
default & safe

→

Execute
release EIP · stop EC2

→

Verify + log
result & rollback

Reversible first

Prefers safe actions (release IP, stop — not delete).

Dry-run default

Real action only behind an explicit "execute for real".

Rollback plan

Every ticket ships with how to undo it.

Safe by construction

01
Read-only default
Discovery never mutates anything.

02
Least-privilege IAM
Only the actions the tools call.

03
Human approval
No action without an owner's OK.

04
Full audit trail
Every request, decision & result.

Enterprise-ready path: AWS Organizations for multi-account, KMS encryption, and SSO + role-based access replacing the demo's role switch.

From demo to platform

Today
insight · savings · approve · guarded act

→

Next
Cost Explorer live · Compute Optimizer

→

Then
multi-account · SSO/RBAC · scheduling

Deeper detection

Rightsizing, unused NAT/ELB, Lambda & RDS tuning, S3 tiering.

Enterprise integration

Organizations, ticketing/Slack hooks, scheduled scans, DynamoDB store.

Insight you trust, savings you approve

- ✓ Continuous AWS cost visibility & analytics.
- ✓ Real waste found with dollar-value savings — read-only.
- ✓ Portal-based approval: ticket → notify → approve → audit.
- ✓ Guarded execution: dry-run first, reversible actions, rollback.
- ✓ gpt-4o + LangChain + boto3, on Streamlit.

The pitch in one line

A FinOps analyst that never sleeps — and never acts without sign-off.

Run it live: `streamlit run app.py` → Dashboard, Chat, and the Manager Approvals portal.